torch.fft.fft2#

https://pytorch.org/docs/stable/generated/torch.fft.fft2.html

Description:

Computes the 2 dimensional discrete Fourier transform of input. Equivalent to fftn() but FFTs only the last two dimensions by default. The Fourier domain representation of any real signal satisfies the Hermitian property: X[i, j] = conj(X[-i, -j]). This function always returns all positive and negative frequency terms even though, for real inputs, half of these values are redundant. rfft2() returns the more compact one-sided representation where only the positive frequencies of the last dimension are returned. Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Function Signature

```
torch.fft.fft2(input, s=None, dim=(-2, -1), norm=None, *,
out=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> x = torch.rand(10, 10, dtype=torch.complex64)
>>> fft2 = torch.fft.fft2(x)
```

```
>>> two_ffts = torch.fft.fft(torch.fft.fft(x, dim=0), dim=1)
>>> torch.testing.assert_close(fft2, two_ffts,
check_stride=False)
```

Notes & Warnings

NOTE Note The Fourier domain representation of any real signal satisfies the Hermitian property: X[i, j] = conj(X[-i, -j]). This function always returns all positive and negative frequency terms even though, for real inputs, half of these values are redundant. rfft2() returns the more compact one-sided representation where only the positive frequencies of the last dimension are returned.

NOTE Note Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Detailed Documentation

Documentation

Computes the 2 dimensional discrete Fourier transform of input. Equivalent to fftn() but FFTs only the last two dimensions by default.

The Fourier domain representation of any real signal satisfies the Hermitian property: X[i, j] = conj(X[-i, -j]). This function always returns all positive and negative frequency terms even though, for real inputs, half of these values are redundant. rfft2() returns the more compact one-sided representation where only the positive frequencies of the last dimension are returned.

Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

input (Tensor) – the input tensor

s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension dim[i] will either be zero-padded or trimmed to the length s[i] before computing the FFT. If a length -1 is specified, no padding is done in that dimension. Default: s = [input.size(d) for d in dim]

dim (Tuple[int], optional) – Dimensions to be transformed. Default: last two dimensions.

norm (str, optional) –

Normalization mode. For the forward transform (fft2()), these correspond to:

"forward" - normalize by 1/n

"backward" - no normalization

"ortho" - normalize by 1/sqrt(n) (making the FFT orthonormal)

Where n = prod(s) is the logical FFT size. Calling the backward transform (ifft2()) with the same normalization mode will apply an overall normalization of 1/n between the two transforms. This is required to make ifft2() the exact inverse.

Default is "backward" (no normalization).

out (Tensor, optional) – the output tensor.

The discrete Fourier transform is separable, so fft2() here is equivalent to two one-dimensional fft() calls: